Plagiarism Detector .net

Plagiarism Detector Report Page

Plagiarism Score	Unique Score	AI Detection Score
6%	94%	?
6% Exact Match		AI detection not performed. Scan with Plagiarism Detector to view results.
0% Partial Match		
Partially Original		

Total Words	986	Total Characters	7743	Plagiarized Sentences	63	Unique Sentences	100%

Plagiarism Sources

References

Document Content

3.3 Study of Proposed System The proposed system combines multi-agent conversational reasoning with automated vulnerability detection. Rather than relying only on single-agent models or isolated static checks, the model evaluates the semantic and logic integrity of smart contracts for vulnerable execution paths. The framework uses data collected through empirical security benchmarks in Code4rena and Ethereum mainnet. The study examines contract functions and state variable transitions Department of Computer Science and Design,FISAT 10Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems characteristics such as reentrancy paths, call execution flow, state update ordering, and authorization barriers. These characteristics are used to assess whether individual functions of the contract satisfy the security specifications of robust decentralized finance. The collected vulnerability information is incorporated into the multi-agent consensus process. This enables the model to identify execution paths that are dangerous for the targeted contract category. The model first seeks the most direct exploit path that satisfies the vulnerability constraints. When a fully verifiable exploit cannot be identified, it provides an alternative audit finding with minimal severity flags. The paper also presents a prototype multi-agent framework to support the practical auditing of code. The application is intended to communicate vulnerability findings based on collaborative decision-making, making the model more practical for smart contract developers. The overall approach connects contract ingestion, multi-agent analysis, and report generation in a single framework. 3.4 Requirement Specification The proposed auditing framework involves contract data collection, semantic vulnerability assessment, agent communication network, consensus evaluation, and the presentation of audit recommendations. Therefore, suitable data, processing capabilities, and software components are required to support the workflow. The study depends on the availability of reliable contract AST information about the target protocol. The collected data must describe the relevant security characteristics and support the evaluation of contracts for different vulnerability categories. A suitable computational environment is also required to process the agent dialogues and calculate consensus scores. 3.4.1 Recommended System Requirements The following requirements describe the general resources needed to support an automated multi-agent auditing framework. They should not be interpreted as exact hardware specifications reported by the paper: • Processor: A modern multi-core processor capable of handling concurrent agent reasoning data and token calculations. • Memory: Sufficient RAM to store and process contract logic information and benchmark audit data. • Storage: Adequate storage for contract source records, vulnerability attributes, benchmark datasets, and token usage information. • Location and Network Data: Stable internet connection for accessing remote LLM model APIs and blockchain testnets. • Execution Environment: A workstation capable of running Python 3.12 and Solidity compilation tools for accessing the prototype application and viewing audit summaries. Department of Computer Science and Design,FISAT 11Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems • Network Connectivity: Connectivity required by the application for accessing cloud LLM endpoints and associated web3 services. 3.4.2 Software Requirements The framework requires software components capable of processing smart contract ASTs, representing the agent conversation network, calculating vulnerability scores, and presenting the final audit results: • Data Processing: Software for storing, organising, and processing the vulnerability audit information. • Agent Network Representation: A suitable representation of conversational nodes and message exchanges with associated prompt attributes. • Auditing Component: A multi-agent consensus mechanism capable of evaluating vulnerability candidates based on BoT scenario templates and identifying genuine exploits. • Vulnerability Assessment: Software logic for evaluating vulnerability characteristics such as state variable updates, external calls, unchecked transfers, and authorization checks. • Mobile/Web Application: An application interface for presenting vulnerability warnings to developers. 3.5 Functional Requirements The accessibility and security routing framework should satisfy the following functional requirements: 1. The system shall use contract source and AST data collected through verified code repositories. 2. The system shall represent smart contract functions and logic blocks in a form suitable for agent debate. 3. The system shall consider reentrancy guards and execution ordering when assessing vulnerability risks. 4. The system shall consider access control design, signature verification, and delegatecall operations as security-critical infrastructure characteristics. 5. The system shall support vulnerability evaluation for common vulnerabilities, complex logic errors, and newly emerged CVEs. 6. The system shall identify the highest-confidence vulnerability exploit path when a vulnerable condition is present. 7. The system shall identify an alternative finding with minimal severity flags when a severe exploit is absent. Department of Computer Science and Design,FISAT 12Advanced Smart Contract Vulnerability Detection via LLM-Powered Multi-Agent Systems 8. The system shall provide actionable remediation recommendations between an identified bug and its patch. 9. The system shall support the presentation of audit

recommendations through the prototype multi-agent application. 3.6 Non-Functional Requirements The smart contract vulnerability detection framework should satisfy the following non-functional requirements: • Usability: Route recommendations and vulnerability warnings should be presented clearly so that smart contract developers can understand and use the information. • Accessibility: The navigation and reporting interface should consider the needs of users with different security and Solidity development expertise. • Performance: The system should calculate suitable audit findings and consensus scores within a practical time. • Reliability: Audit recommendations should be based on consistent processing of the available contract logic data. • Accuracy: Vulnerability assessments should reflect the actual security characteristics recorded during the audit benchmarks. • Adaptability: The framework should accommodate different vulnerability categories and Solidity compiler versions for the supported categories of contracts. • Maintainability: Contract vulnerability records and prompt templates should be organised so that updates can be incorporated into the auditing process. • Safety and Inclusiveness: The system should support safer and more trustworthy smart contract deployment without assuming that a recommended contract is completely bug-free when complex logic anomalies remain. 3.7 Summary This chapter analysed the proof of concept, existing system, proposed system, and requirements of the multi-agent vulnerability detection framework for smart contract developers. The reviewed framework addresses the limitations of conventional static analysis by considering deep semantic code characteristics and the diverse requirements of decentralized application auditors. The proposed approach connects smart contract AST parsing with multi-agent collaborative debate and automated report generation. Its effectiveness depends on the quality and completeness of the available thought templates and LLM context depth. Overall, the framework provides an approach to improving smart contract security by incorporating multi-agent collaborative reasoning into decentralized software development. Department of Computer Science and Design,FISAT